Sets and Sorting

Lecture 03 and 04

Ari, October 7th, Wednesday, 2026, NYU-Lond

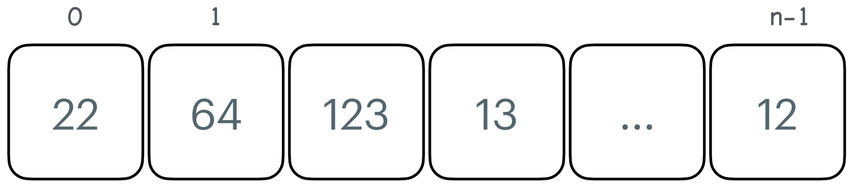
Review

- Last time we introduced this idea of the Interface and the Data Structure.
- Loosely, the Interface told us what the problem we were trying to solve was.
- The data structure told us how to solve the problem.
- We introduced one interface, the Sequence interface.

The Static Sequence Interface

- build(X)
- len()
- iter_seq()
- get_at(i)
- set_at(i)

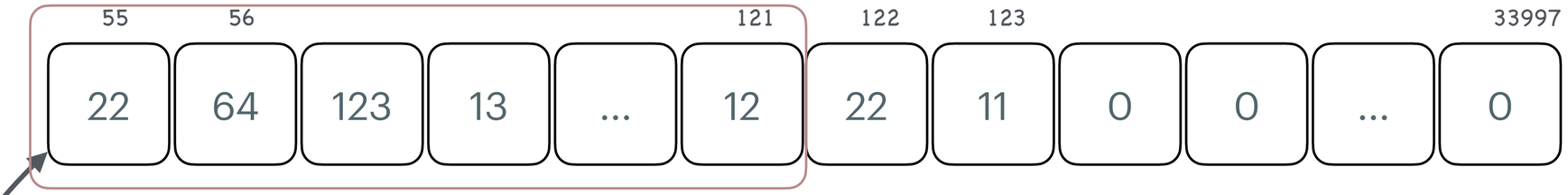
Some data X whose ordering matters

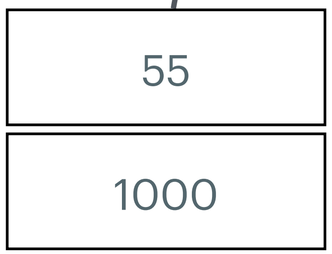


Solution: Arrays



Solution: Arrays



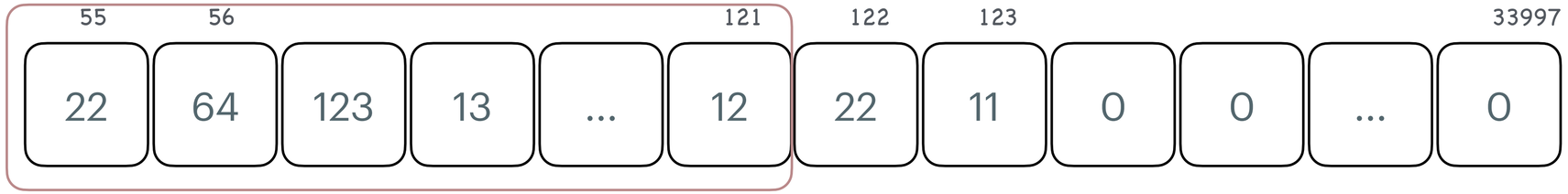


Addr array starts at

Length of array

Can ask for 1000 contiguous cells of memory in one go takes O(1000) time.

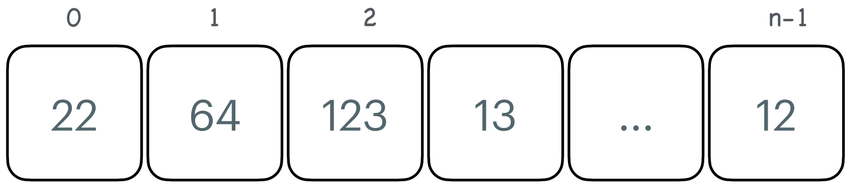
Solution: Linked Lists





Dynamic Sequence Interface

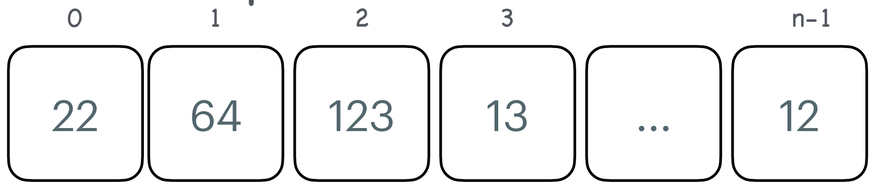
- build(X)
- len()
- iter_seq()
- get_at(i)
- set_at(i)
- insert_at(i, a)
- delete_at(i)



Insert a at position 2



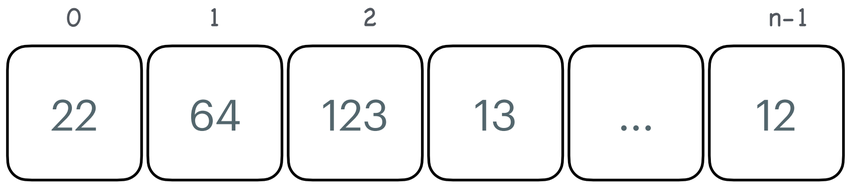
Delete at position 2



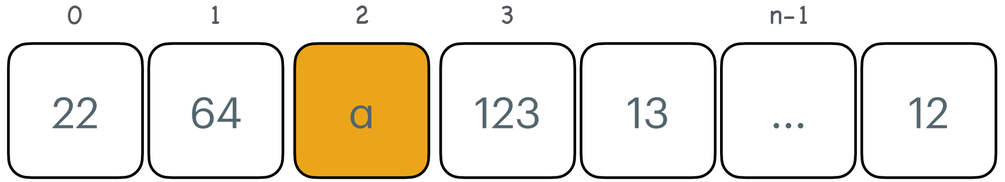
Dynamic because size of arrays are changing

Dynamic Sequence Interface

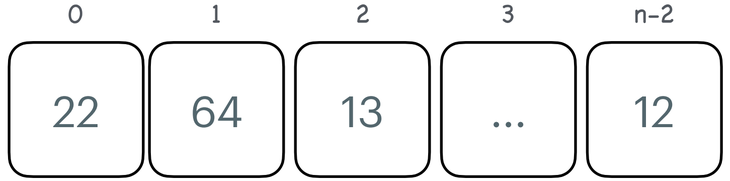
- build(X)
- len()
- iter_seq()
- get_at(i)
- set_at(i)
- insert_at(i, a)
- delete_at(i)



Insert a at position 2



Delete at position 2





Dynamic because size of arrays are changing

Summary so far

	build	len	get_at(i)	set_at(i)	insert_at(i, a)	delete_at(i)	ins_first(a)	ins_last(a)	delete_first()	delete_last()
Static Arrays	O(n)	O(1)	O(1)	O(1)	O(n)	O(n)	O(n)	O(n)	O(n)	O(1)
Linked Lists	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)	O(1)	O(1)	O(1)	O(1)

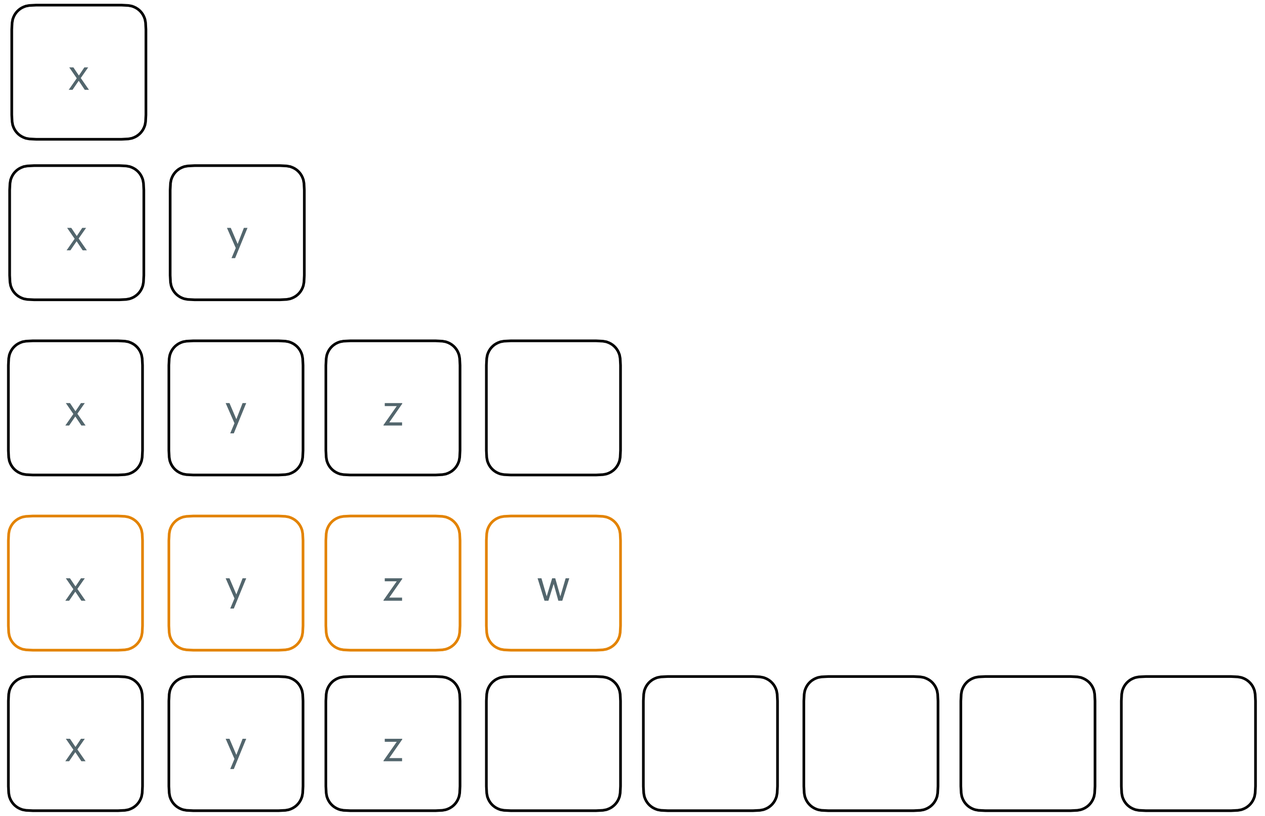
Assuming we have a tail pointer

Dynamic Array

- Key Idea: If I need to store n things in an array ask for space 2n.
- Remember a static array only asks for n cells to store n things.
- This means there's usually room at the end but sometimes not, and we showed that this leads to amortised time complexity O(1) for `insert_last(a)`

Dynamic Array

- Key Idea: If I need to store n things in an array ask for space 2n.
- Remember a static array only asks for n cells to store n things.
- This means there's usually room at the end but sometimes not, and we showed that this leads to amortised time complexity O(1) for insert_last(a)



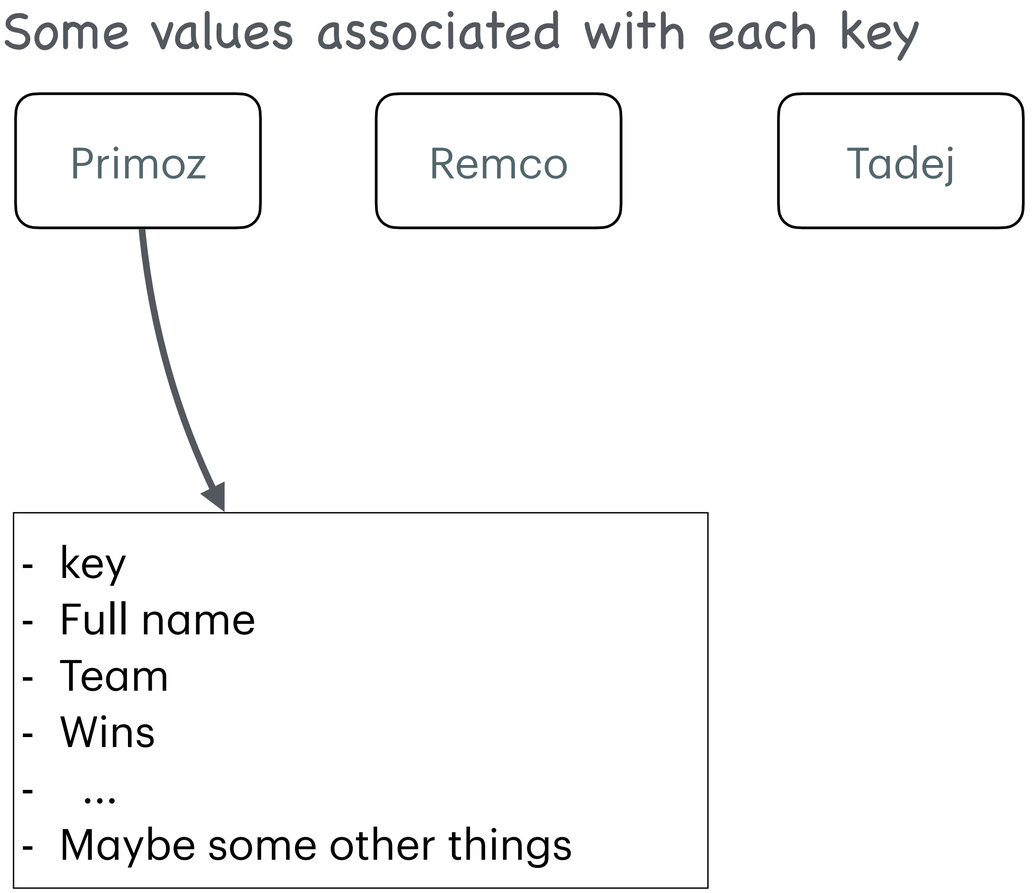
n inserts take $1 + 2 + 4 + 8 \ldots = \sum_{i=0}^{\log n} 2^i = 2^{\log n + 1} - 1 = O(n)$

Summary so far

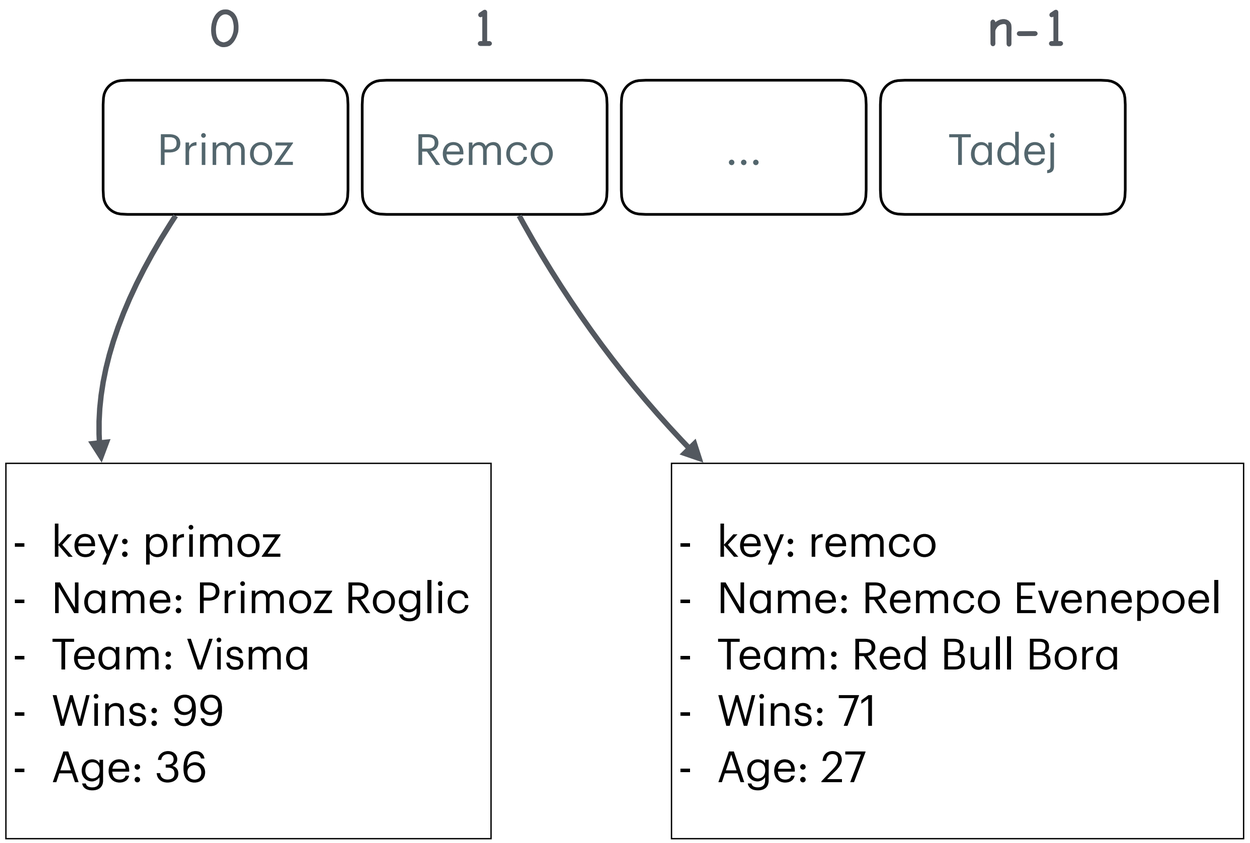
	build	len	get_at(i)	set_at(i)	insert_at(i, a)	delete_at(i)	ins_first(a)	ins_last(a)	delete_first()	delete_last()
Static Arrays	O(n)	O(1)	O(1)	O(1)	O(n)	O(n)	O(n)	O(n)	O(n)	O(1)
Linked Lists	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)	O(1)	O(1)	O(1)	O(1)
Dynamic	O(n)	O(1)	O(1)	O(1)	O(n)	O(n)	O(n)	O_T(1)	O(n)	O(1)

Set Interface (A collection Of Things)

- build(X)
- len()
- find(key)
- insert(object) : Insert object by its key
- delete(key)
- find_min(), find_max() (Some ordering over keys)
- find_next(key), find_prev(key) (over the same order)



Use An Array (This is not a good idea)

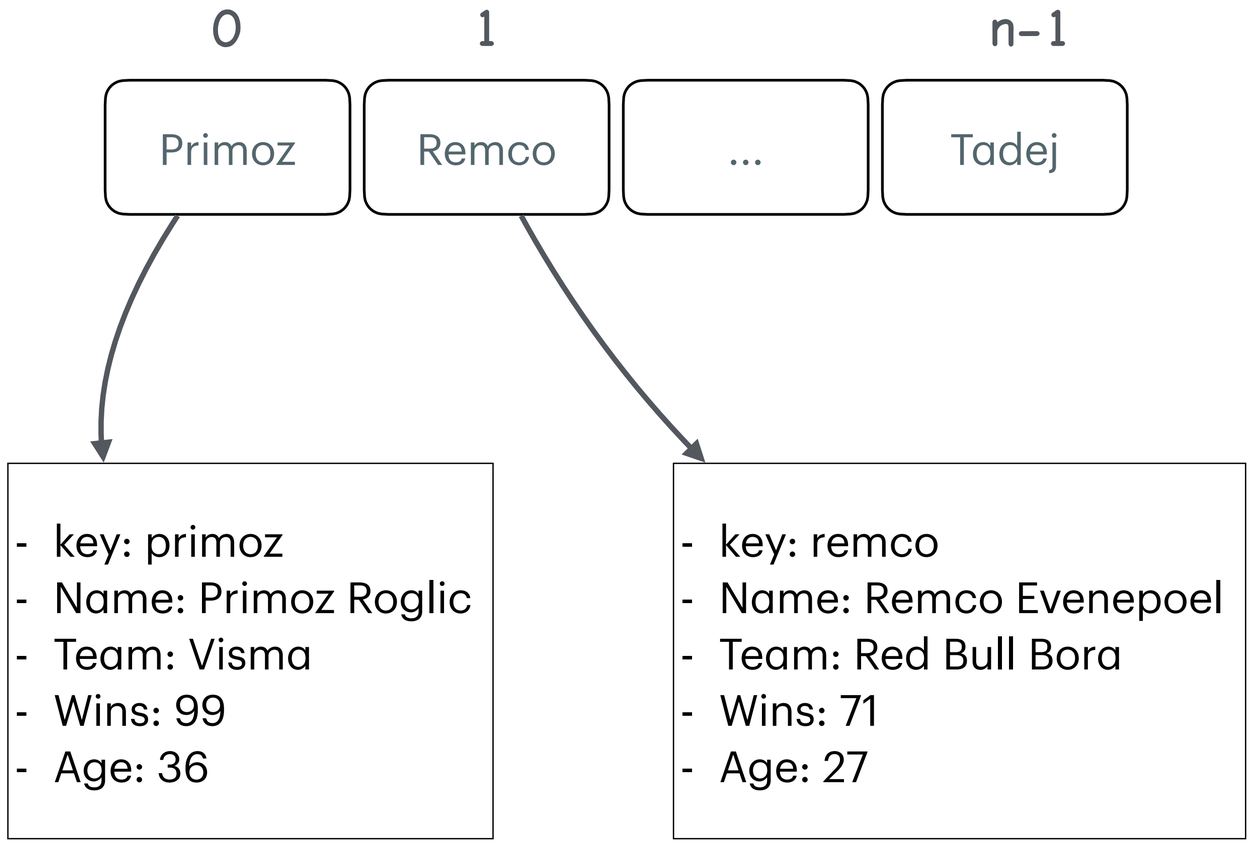


find("tadej") takes O(n) time

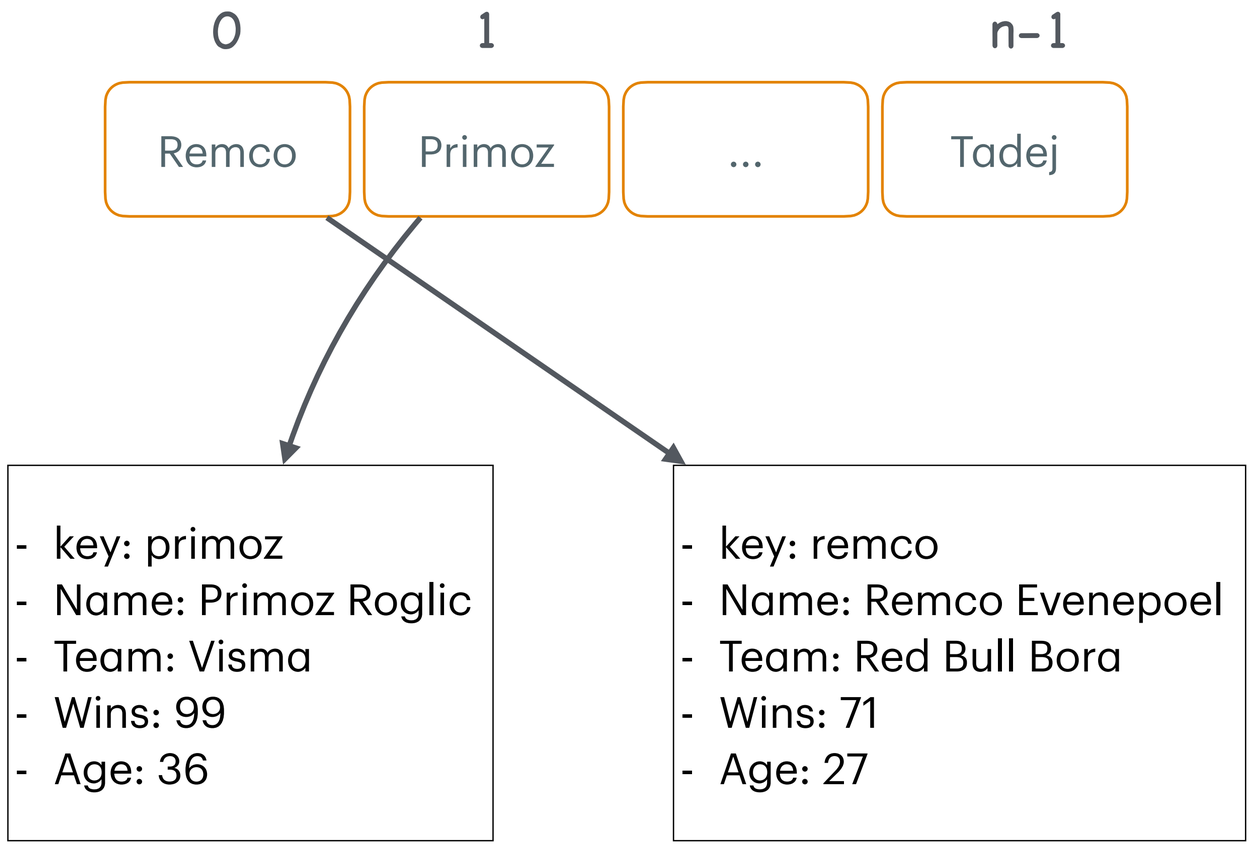
Complexity

				Dynamic	Ordering on keys	
	build	**len**	**find(key)**	**Insert(x)** **delete(k)**	**find_min()** **find_max()**	**find_prev(key)** **find_next(key)**
Arrays	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)

Attempt Two: Sorted Arrays



Attempt Two: Sorted Arrays



Binary Search

BinarySearch(A, target, 0, length(A) - 1)

```
BinarySearch(A, target, low, high)
    if low > high
        return -1
    mid ← low + floor((high - low) / 2)
    if A[mid] = target
        return mid
    else if target < A[mid]
        return BinarySearch(A, target, low, mid - 1)
    else
        return BinarySearch(A, target, mid + 1, high)
```

Hw2: Show that it takes time $\Omega(logn)$

Complexity

	build	len	find(key)	Insert(x) delete(k)	find_min() find_max()	find_prev(key) find_next(key)
Arrays	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Sorted Arrays	O(n log n)	O(1)	log (n)	~~O(n log n)~~ O(n)	O(1)	O(log n)

Show them why asymptotic matter

Hw2: Implement the find on a sorted and unsorted array for different values of **n**

So how do we sort?

- Some vocabulary:
  - Sort is destructive if it overwrites the input.
  - Sort is in place if only need O(1) space to sort.

Permutation Sort.

- Input a : List of n numbers, strings



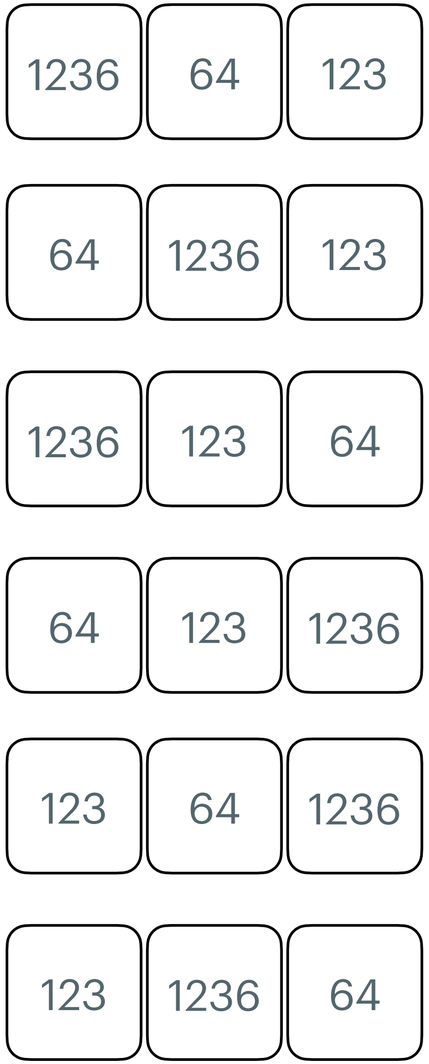
- Compute list of all permutations of a
- Now go through each permutation,
  - check if permutation is sorted
  - if yes -- it's right answer

Permutation Sort.

- Input a : List of n numbers, strings



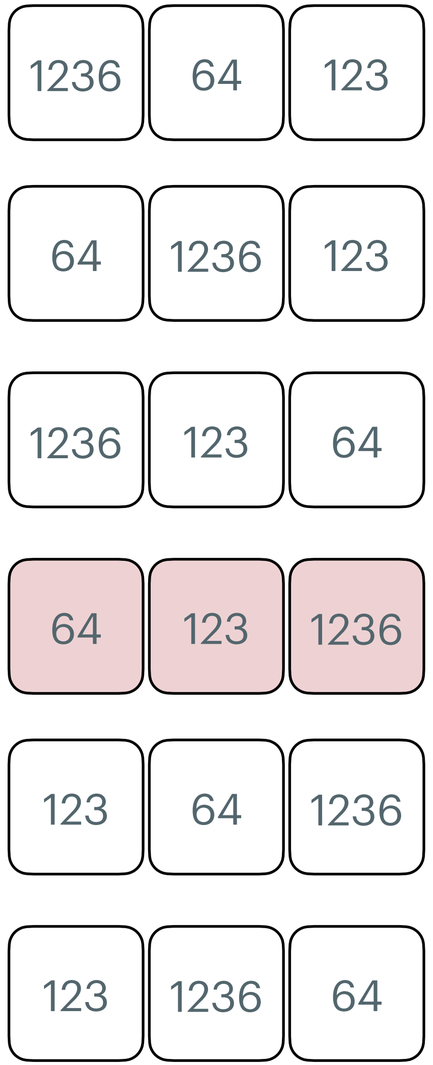
- Compute list of all permutations of a
- Now go through each permutation,
  - check if permutation is sorted
  - if yes -- it's right answer



Permutation Sort.

- Input a : List of n numbers, strings
- Compute list of all permutations of a
- Now go through each permutation,
  - check if permutation is sorted
  - if yes -- it's right answer





Time Complexity: $\Omega(n!)$

Selection Sort



Each time find the largest item, and put it at the end of the list

Selection Sort



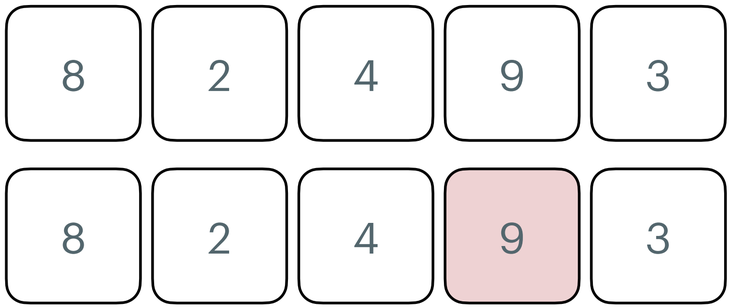
Each time find the largest item, and put it at the end of the list

Selection Sort



Each time find the largest item, and put it at the end of the list

Selection Sort



Each time find the largest item, and put it at the end of the list

Selection Sort



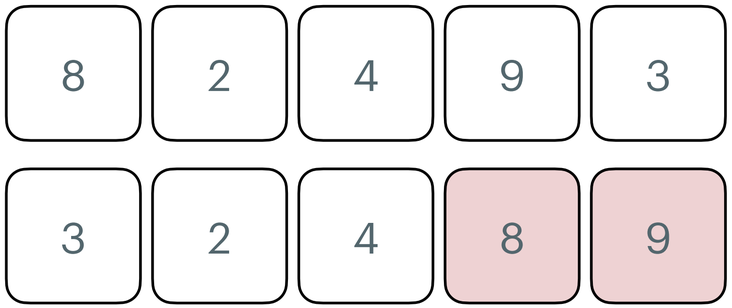
Each time find the largest item, and put it at the end of the list

Selection Sort



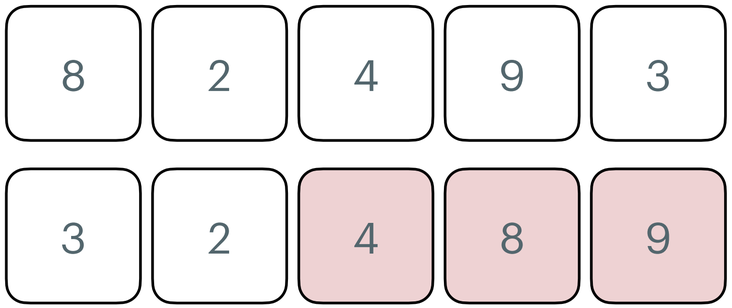
Each time find the largest item, and put it at the end of the list

Selection Sort



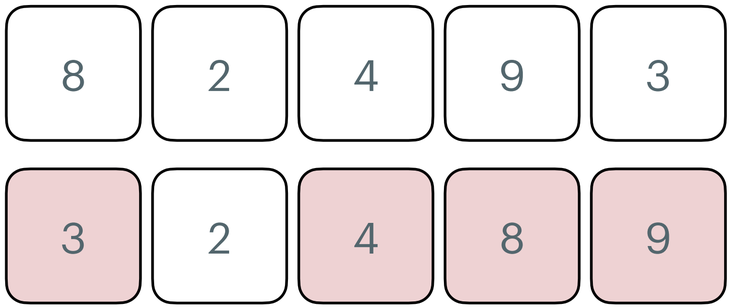
Each time find the largest item, and put it at the end of the list

Selection Sort



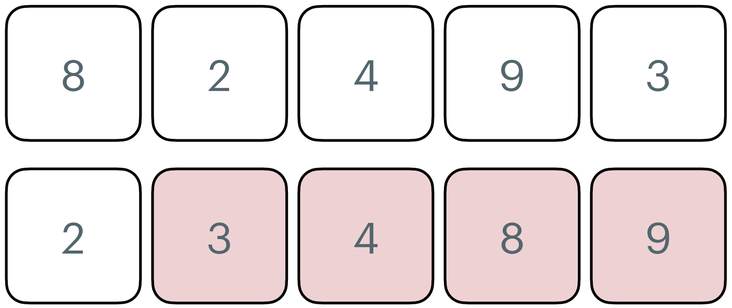
Each time find the largest item, and put it at the end of the list

Selection Sort



Each time find the largest item, and put it at the end of the list

Selection Sort



List is sorted!

Each time find the largest item, and put it at the end of the list

How Does One Prove This Algorithm Is Correct?

- Mathematical Tool - Induction
- General idea:
  - Find biggest element index <= i (prefix_max)
    - Run time $S(n) = \theta(n)$
  - Swap with i
  - Sort the remainder of the list
- Total runtime = $O(n^2)$

selection_sort.py

```python
def prefix_max(A, i):
    if i > 0:
        j = prefix_max(A, i-1)
        if A[j] > A[i]:
            return j

    return i

def selection_sort(A, i = None):
    """Sort A
    """
    if i is None: i = len(A) - 1
    if i > 0:
        j = prefix_max(A, i)
        A[i], A[j] = A[j], A[i]
        selection_sort(A, i-1)

if __name__ == "__main__":

    A = [8, 1, 4, 5, 3]
    selection_sort(A)
    print(A)
```

Prefix Max Analysis

- Base case : i = 0, there is 1 element in array, it is maximal by defn
- Assume for i = n-1, prefix_max gives us the maximal element.
- For i = n, two things can happen, the n'th thing is bigger, or the previous max was bigger.
- So we return the max of the the two.

Run Time

- $T(n) = T(n-1) + \theta(n)$
- $T(1) = O(1)$

Insertion Sort

- Homework Problem. The opposite idea, find the smallest item.

Merge Sort



Claim the array is not in sorted order.

But every individual cell if that was the array is in sorted order.

Merge Sort



This array is clearly not sorted

But I claim that each individual cell is sorted.

Merge Sort



This array is clearly not sorted

But I claim that each individual cell is sorted.

Now what about the problem with two cells

Merge Sort

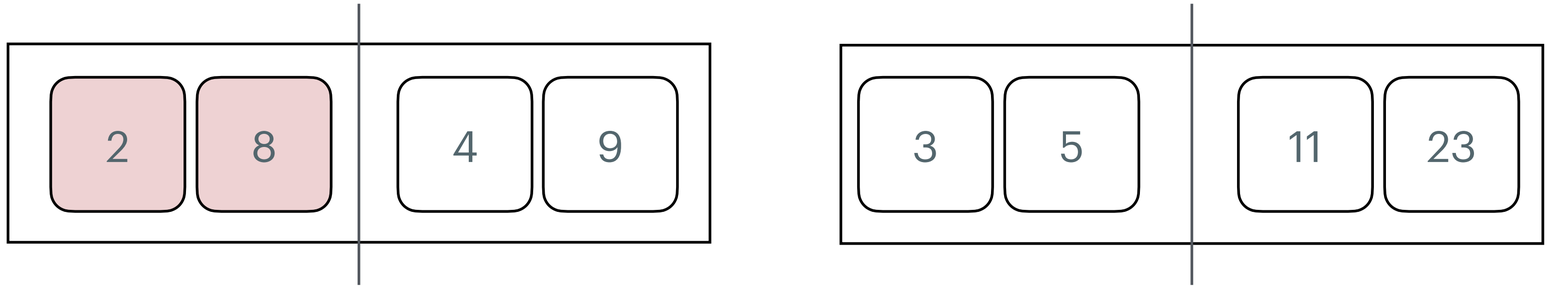


This array is clearly not sorted

But I claim that each individual cell is sorted.

Now what about the problem with two cells - merge them.

Merge Sort



This array is clearly not sorted

But I claim that each individual cell is sorted.

We know when do this merge, the left and right sides are already sorted.

Do the two finger algorith on the board

Merge Sort

```
MergeSort(A)
    if length(A) ≤ 1
        return A

    mid ← floor(length(A) / 2)

    left ← MergeSort(A[0 : mid])
    right ← MergeSort(A[mid : length(A)])

    return Merge(left, right)
```

Time complexity?

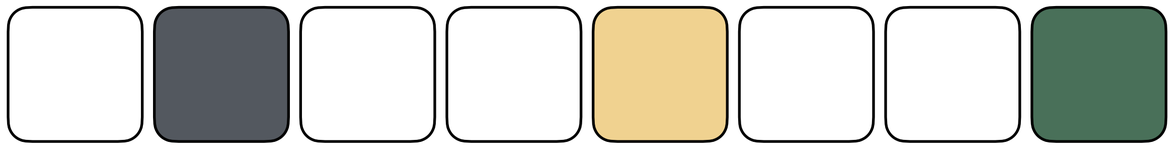
Hashing

	build	len	find(key)	Insert(x) delete(k)	find_min() find_max()	find_prev(key) find_next(key)
Arrays	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Sorted Arrays	O(n log n)	O(1)	log (n)	~~O(n log n)~~ O(n)	O(1)	O(log n)

Still not quite satisfied

Direct Access Arrays (Prelude)

- Pretend our keys were numbers.
- Ask for an array the size of the universe.
- Store the key there.
- Can only have one unique key but that is ok

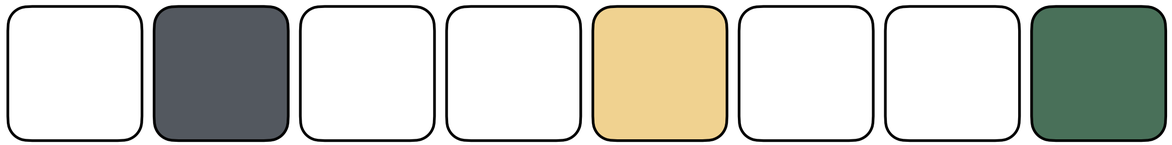


Story

	build	len	find(key)	Insert(x) delete(k)	find_min() find_max()	find_prev(key) find_next(key)
Arrays	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Sorted Arrays	O(n log n)	O(1)	log (n)	~~O(n log n)~~ O(n)	O(1)	O(log n)
Direct Access	O(U)	O(1)	O(1)	O(1)	O(U)	O(U)

Space Problem : Soln Ask For Less Space

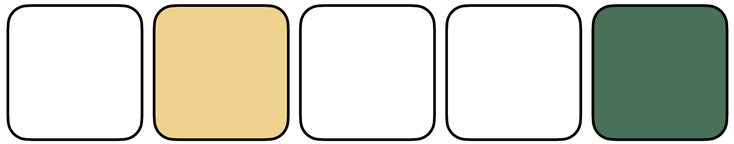
- Ask for m cels.



$h : [U] \to [m]$

$m = \theta(n)$

But collisions??



How To Fix Collisions

- Open Addressing : Leave a little space, 2n? (Hard to analyse)
- Or Store a pointer to another data-structure (chains)
- How to make chains short?

Chains Short: Division

- $h(k) = k \mod m$
- But need input distribution assumption.
- Exists adversarial inserts. (Python3) does.
- Is this good in theory? Random?
- Space of hash functions $m^{|U|}$ (Computationally hard)

Universal Hash Functions

- $h_{ab}(k) = ((ak + b) \mod p) \mod m)$
- $\mathcal{H} = \{h_{ab}\}_{a,b \in [p], a \neq 0}$
- What is a Universal Hash?
- $\Pr_{h \leftarrow \mathcal{H}} \left[h(k_1) \neq h(k_2) \right] \leq 1/m$
  - Its saying any two keys collide with probability 1/m, as if the collisions were uniformly random for two keys.x§

Hw2: Prove that this is a universal hash function

$$k \in U, \quad h(k) \in M$$

Size of chains when using Universal

$X_{i,j}$: Does key $h(k_i)$ and $h(k_j)$ collide?

Expected number of collisions for bucket i

$$\mathbb{E}(S_i) = \mathbb{E}\sum_{j=1}^{n} X_{i,j} = \sum_{j=1}^{n} \mathbb{E}X_{i,j} = 1 + \sum_{j \neq i} \mathbb{E}X_{i,j} = 1 + (n-1) \Pr_{h \leftarrow \mathcal{H}}[h(k_i) = h(k_j)] \leq 1 + \frac{n-1}{m}$$